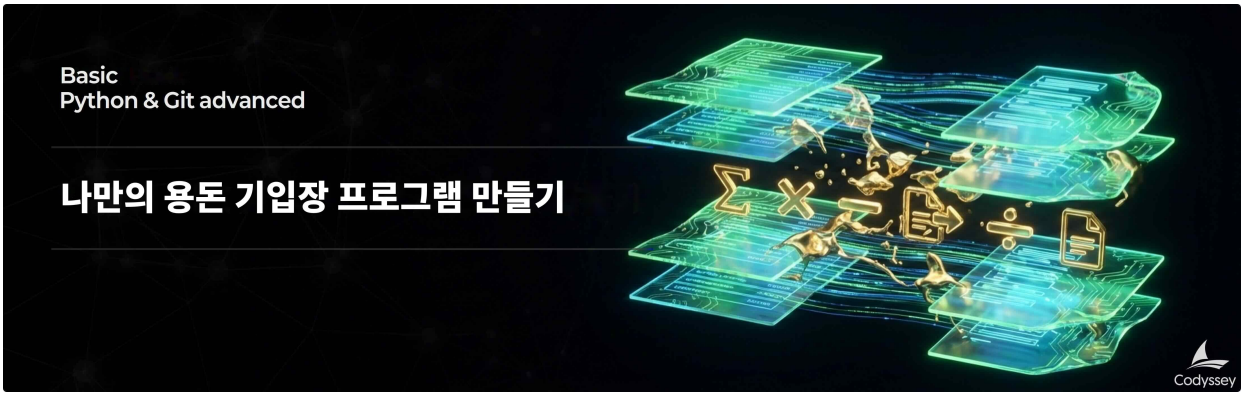
분야 AI/SW 기초 구분 Python과 Git 심화 학습시간 60시간 필수/선택 필수

나만의 용돈 기입장 프로그램 만들기

문제기술

기술적 설명



1. 미션 소개

'작은 서비스'란 기능이 많은 게 아니라 예외 상황에서도 데이터가 안전한 것을 말합니다. 이번에 만드는 건 콘솔 가계부인데, 제너레이터 스트리밍, 데코레이터 분리, 타입 힌트까지 구조를 제대로 챙깁니다. 단순한 프로그램이 아니라, 유지보수 가능한 설계로 완성한다는 관점으로 접근하세요.

이번 미션은 Python으로 파일 입출력 기반의 가계부 콘솔 프로그램을 구현하는 과제입니다. 수입과 지출 내역을 단순히 저장하는 수준을 넘어, 수정/삭제, 검색, 월별 요약, 카테고리 관리, 예산 초과 경고까지 포함한 "작은 서비스" 형태로 완성합니다.

학습자는 터미널에서 명령어를 입력해 내역을 추가하고(add), 목록을 조회하며(list), 특정 조건으로 검색하고(search), 월별 요약과 카테고리 리포트를 출력하고(summary), 데이터를 내보내고(export), 기존 파일을 가져오는(import) 기능을 구현합니다. 데이터는 프로그램 종료 후에도 유지되도록 JSONL 또는 CSV 파일로 영구 저장해야 합니다.

또한, 저장 파일을 한 번에 모두 읽지 않고 제너레이터로 스트리밍 처리하며, 데코레이터로 공통 기능(예외 처리, 실행 로그, 실행 시간 측정 등)을 분리합니다. 타입 힌트를 적용해 함수와 데이터 구조의 계약을 명확히 하고, 모듈 분리로 유지보수 가능한 구조를 설계합니다.

2. 최종 결과물

다음 10가지 기능이 정상 동작하는 애플리케이션 1개를 완성한다.

1. 거래 추가(add)

- 입력/요청: 대화형 입력(날짜, 타입, 카테고리, 금액, 메모/태그)
- 출력/화면: 저장 성공 메시지 + 생성된 거래 id

2. 거래 목록(list)

- 입력/요청: -limit 등 옵션
- 출력/화면: 최신순 거래 리스트(스트리밍 처리)

3. 거래 검색(search)

- 입력/요청: -from/--to , -category , -type , -q , -tag
- 출력/화면: 조건에 맞는 거래 리스트(최신순)

4. 월별 요약(summary)

- 입력/요청: -month YYYY-MM , -top N
- 출력/화면: 총수입/총지출/잔액 + 카테고리별 지출 TOP N

5. 예산 설정/조회(budget)

- 입력/요청: budget set --month YYYY-MM --amount 금액
- 출력/화면: 저장 성공 메시지 + summary에서 예산 사용률/초과 경고

6. 카테고리 관리(category)

- 입력/요청: category add/list/remove
- 출력/화면: 카테고리 목록/추가/삭제 결과(사용 중 카테고리 처리 포함)

7. 거래 수정(update)

- 입력/요청: -id 기반(옵션 방식 또는 대화형 중 1개로 고정)
- 출력/화면: 수정 성공/실패 메시지(없는 id 처리 포함)

8. 거래 삭제(delete)

- 입력/요청: delete --id <id>
- 출력/화면: 삭제 성공/실패 메시지(없는 id 처리 포함)

9. 가져오기/내보내기(import/export)

- 입력/요청: import --from <csv> , export --out <csv> --month ... 또는 -from/--to
- 출력/화면: 처리 건수 출력 + CSV 파일 생성/반영 확인

10. **추가 조건(최종 결과물의 필수 구성)**
    * 데이터는 3개 이상 파일로 영구 저장되어야 한다. (예: transactions / categories / budgets)
    * README.md 에 실행 방법, 저장 파일 위치/형식, 주요 명령 예시, import/export CSV 스키마가 포함되어야 한다.

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- 파일 기반 저장(JSONL/CSV)으로 데이터를 영구 저장하고, CRUD/검색/요약/입출력을 구현할 수 있다.
- 콘솔 프로그램을 클래스/모듈로 구조화하고, 각 계층(모델/저장소/서비스/CLI)의 책임을 설명할 수 있다.
- yield 기반 제너레이터로 대용량 파일도 스트리밍 처리하는 이유와 동작 방식을 설명할 수 있다.
- 데코레이터로 공통 관심사(로그/예외/시간 측정) 를 분리한 구조와 이유를 설명할 수 있다.
- 타입 힌트를 통해 입출력 계약을 명확히 했을 때 얻는 이점을 실제 코드 예로 설명할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 실행 및 입력 방식

- 실행 예시는 아래 중 하나를 권장합니다.
    - python -m budget_app <command> [options]
- 모든 명령은 --help 옵션으로 사용 방법이 출력되어야 합니다.
- 입력 기본 방식은 “대화형” 입니다.
    - 예: add 실행 시 날짜/타입/카테고리/금액 등을 input() 으로 순차 입력

- 단, 아래 항목은 옵션 인자 방식도 허용(권장) 합니다.
    - search , list , summary , export , import , delete
    - update 는 “옵션 방식 또는 대화형” 중 하나를 선택해도 되나, 문서에 명확히 고정해야 합니다(아래 6번 참고).
- 옵션 표기는 리눅스 표준인 -- 로 통일해야 합니다.
    - 예: --help , --limit , --from , --to , --month

2. 데이터 모델

- 거래 내역(Transaction)은 최소 아래 필드를 포함해야 합니다.
    - id (유일), type (income/expense), date (YYYY-MM-DD), amount (양수), category , memo (선택), tags (선택)
- 데이터 모델은 dataclass 또는 그에 준하는 구조로 정의해야 합니다.
- 최소 2개 이상의 클래스를 사용해야 합니다.
    - 예시: Transaction , TransactionRepository , BudgetStore , CategoryStore , BudgetService 등
- 입력 검증
    - 날짜 형식 오류, 음수/0 금액, 허용되지 않은 type, 존재하지 않는 category 등은 재입력 요구 또는 오류 메시지 출력으로 처리합니다.

3. 저장 정책

- 저장 포맷은 JSONL 또는 CSV 중 1개를 선택합니다.
- 저장 파일은 3개 이상(필수) 로 분리해 영구 저장합니다.
    - transactions.<fmt> , categories.<fmt> , budgets.<fmt>
- 기본 저장 폴더는 ./data 를 권장하며, 옵션으로 변경 가능해야 합니다(예: -data-dir).
- 초기 실행(저장 파일이 없을 때)
    - 파일이 없으면 자동 생성하거나, “초기화 안내 메시지”를 출력해야 합니다.
    - 카테고리 파일이 비어있으면 아래 중 하나를 택해 동작을 명확히 하세요.

    - (안 A) 기본 카테고리 자동 생성(예: food, transport, rent, etc)
    - (안 B) category add 를 먼저 하도록 안내하고 add 를 막음

4. add(거래 추가)

- add 실행 후 대화형으로 필드를 입력받아 거래를 저장해야 합니다.
- category는 등록된 목록에 존재해야 합니다(없으면 안내 후 재입력/등록 유도).
- 저장 완료 시 생성된 id 를 사용자에게 출력해야 합니다.

5. list(목록 조회)

- list 는 최신순으로 거래를 출력해야 합니다.
- --limit N 옵션을 지원해야 합니다(기본값 제공).
- 파일 전체를 한 번에 로드하지 않고, 제너레이터 기반 스트리밍 처리로 구현해야 합니다.

6. update/delete(수정/삭제)

- delete --id <id> 로 특정 거래를 삭제할 수 있어야 합니다.
- 존재하지 않는 id는 “없는 데이터”로 처리하고 사용자 메시지를 출력해야 합니다.
- update 는 아래 중 하나의 방식으로 문서에 명확히 고정해 구현해야 합니다.
    - (안 A) 옵션 기반: update --id <id> [--date ...] [--type ...] [--category ...] [--amount ...] [--memo ...] [--tags ...]
    - (안 B) 대화형 기반: 수정할 필드만 선택/재입력 받는 흐름
- 파일 기반 저장에서 update/delete는 “전체 재작성/임시 파일/원자적 교체(권장)” 등 안정성을 고려해야 합니다.

7. search(검색)

- 조건 기반 검색을 지원해야 합니다.
    - 기간: --from , --to
    - 카테고리: --category
    - 타입: --type
    - 메모 키워드: --q
    - 태그: --tag

- 검색 결과는 최신순으로 출력해야 합니다.
- 스트리밍 처리(제너레이터 기반)를 유지해야 합니다.

8. summary(월별 요약)

- summary --month YYYY-MM 입력을 받아 해당 월의 요약을 출력해야 합니다.
- 출력 항목:
  - 총 수입, 총 지출, 잔액(총수입-총지출)
  - 카테고리별 지출 합계 TOP N(-top 옵션 지원)
- 데이터가 없는 달은 “데이터 없음”을 명확히 출력해야 합니다.

9. budget(예산)

- budget set --month YYYY-MM --amount <금액> 으로 월 예산을 저장해야 합니다.
- summary 실행 시 예산이 설정되어 있으면
  - 예산 대비 사용률(%)
  - 초과 여부(초과 시 경고 문구) 를 함께 출력해야 합니다.
- 예산 데이터도 반드시 영구 저장되어야 합니다.

10. **category(카테고리 관리)**
    * category add/list/remove 를 제공해야 합니다.
    * 카테고리 삭제 시, 해당 카테고리를 사용하는 내역이 존재하면
    * 삭제를 막거나
    * 대체 카테고리를 요구해야 합니다
11. import/export(가져오기/내보내기)
    * import --from \로 거래를 일괄 등록한다.
    * export --out \로 조건에 맞는 거래를 CSV로 저장한다.
    * export는 --month YYYY-MM 또는 --from YYYY-MM-DD --to YYYY-MM-DD 중 하나 이상 조건을 필수로 받는다.
    * import/export는 아래 CSV 최소 스키마를 고정한다.
    *

```
CSS
| column | required | 설명 |
| --- | --- | --- |
| date | Y | YYYY-MM-DD |
```

type	Y	income / expense
category	Y	등록된 카테고리
amount	Y	양수 정수
memo	N	문자열
tags	N	쉼표(,) 구분 문자열
공통: UTF-8, 헤더 포함		

1. 데코레이터(Decorator)

    - 공통 관심사(예: 예외 처리/로그/시간 측정) 데코레이터를 1개 이상 구현하고 실제 적용한다.

2. 예외 처리 및 종료 코드

- 오류는 스택트레이스 대신 원인 + 해결 힌트로 출력한다.
- 정상 종료는 0, 오류 종료는 0이 아닌 값으로 종료한다.

3. 모듈화(구조화)

- 한 파일에 몰아넣지 않고 최소 3개 이상 모듈로 분리한다.
- (권장) CLI / 서비스 / 저장소(파일 I/O) / 모델(데이터 구조)로 책임을 나눈다.

5. 보너스 과제 (선택)

1. 백업 기능

- backup 실행 시 타임스탬프가 포함된 백업 파일을 생성한다.
- 배움 포인트: 파일 처리 + 운영 안전장치(복구 가능성)

2. 반복 내역 기능

- 월급/월세처럼 반복되는 내역을 등록하고, 특정 월에 자동 생성한다.
- 배움 포인트: 규칙 기반 데이터 생성 + 예외 처리

3. 출력 포맷 테이블 정렬

- 외부 라이브러리 없이 문자열 정렬로 가독성을 개선한다.
- 배움 포인트: 콘솔 UX + 포맷터 분리

4. 저장 원자성 강화

    - update/delete 시 임시 파일에 쓰고 rename으로 교체하는 방식을 적용한다.
    - 배움 포인트: 파일 기반 트랜잭션/원자성 사고

개발환경

6. 개발 환경

- Python 3.10 이상

제약조건

7. 제약 사항

- 라이브러리
    - 표준 라이브러리만 사용 가능
    - 별도 `pip install` 이 필요한 외부 라이브러리 사용 금지
- 저장 방식
    - JSONL 또는 CSV 중 1개를 선택해 사용
    - 저장 파일은 3개 이상(transactions/categories/budgets)으로 분리
- CLI 규칙
    - 옵션 표기는 `-` 로 통일
- 오류 처리
    - 스택트레이스 출력 금지(원인 + 해결 힌트 출력)
    - 오류 종료 시 exit code는 0이 아니어야 함

Test Case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구와 디자인은 달라도 된다.

- add(거래 추가) 화면:

```CSS
$ python -m budget_app add
날짜(YYYY-MM-DD): 2024-01-15
타입(income/expense): expense
카테고리: food
금액(양수): 15000
메모(선택): 점심
태그(쉼표로 구분, 없으면 엔터): meal
[저장 완료] id=TX-000012
```

- list(거래 목록) 화면:

```CSS
$ python -m budget_app list --limit 3
TX-000012 | 2024-01-15 | expense | food | 15000 | 점심
TX-000011 | 2024-01-14 | income  | salary | 3000000 |
TX-000010 | 2024-01-12 | expense | transport | 20000 |
```

- category(카테고리 관리) 화면:

```CSS
$ python -m budget_app category add
카테고리명: food
[저장 완료] category=food

$ python -m budget_app category list
- food
- transport
```

- budget + summary(예산 + 월별 요약) 화면:

```
$ python -m budget_app budget set --month 2024-01 --amount 500000
[저장 완료] 2024-01 예산 500000원

$ python -m budget_app summary --month 2024-01 --top 3
총 수입: 3000000원
총 지출: 215000원
잔액: 2785000원
예산: 500000원 (사용률 43.0%)

지출 TOP 3
1) rent 150000원
2) food 45000원
3) transport 20000원
```

CSS

- export / import(CSV 내보내기/가져오기) 화면:

```
$ python -m budget_app export --out export.csv --month 2024-01
[완료] export.csv (12 records)

$ python -m budget_app import --from import.csv
[완료] imported=5, skipped=0
```

CSS

- 오류 출력(예시) 화면:

CSS

```
$ python -m budget_app add
날짜(YYYY-MM-DD): 2024-13-40
[오류] 날짜 형식이 올바르지 않습니다 (YYYY-MM-DD).
[힌트] 예: 2024-01-15
```